

INFORME PRELIMINAR – DESAFÍO 1

Implementación de Sweet Crush mediante manipulación de bits en C++

Curso: Informática II

Semestre: 2026-2

Alejandro Macias Rodríguez

CONTEXTUALIZACION:

Este desafío consiste en desarrollar una versión del juego sweet crush mediante manipulación de bits. La idea es que el programa deberá administrar un tablero rectangular compuesto por ocho tipos diferentes de fichas cada una de 3 bits y permita operaciones propias del juego, tales como, eliminación de fichas, detección de combinaciones horizontales y verticales, reorganización de las fichas, generación de nuevas fichas y procesamiento de cascadas.

Además de la lógica del juego, el sistema debe permitir modificar dinámicamente la estructura del tablero agregando o eliminando filas y columnas en cualquier posición válida (no solo en los extremos). Para esto será necesario trabajar con punteros, arreglos, memoria dinámica y operaciones a nivel de bits.

ANALISIS:

Considero que lo mas importante a considerar desde un principio es el hecho de que podríamos afirmar que realmente nunca va a existir como tal un tablero en la memoria, sino que con lo que vamos a trabajar es con una secuencia continua de bits. Si el tablero es de dimensión $F * C$, entonces la cantidad de fichas seria precisamente $F * C$, sin embargo, la cantidad mínima de bytes puede calcularse de la siguiente manera: $(F * C) / 8$.

Para acceder a una ficha será necesario hacer el proceso de pasar de una posición lógica a como tal los bits de la ficha en cuestion. Si la posición solicitada es (fila , columna) entonces:

- índice = fila * columna + columna
- bit inicial = índice * 3
- byte inicial = bit inicial / 8

Ya para este punto aparece el primer problema, y es que pueden pasar 2 cosas, la ficha esta contenida por completo en un byte o esta en contenida en 2 bytes diferentes. Es justamente el segundo caso el que nos genera conflicto, por lo tanto, vamos a tener que usar operaciones a nivel de bits para poder extraer los bits de cada ficha correctamente.

Otro problema importante a resolver es la detección de combinaciones, vendrían siendo 3 o mas fichas iguales ubicadas consecutivamente de manera vertical o horizontal, pero debemos tener un especial cuidado en caso tal de que una ficha este haciendo parte de 2 combinaciones al mismo tiempo.

También se pueden presentar cascadas, la cosa es que siempre que se elimine una ficha debe actualizarse el tablero ingresando fichas verticalmente (como si se tratase de un efecto de gravedad) entonces después de una combinación se eliminan las fichas de dicha combinaciones pero eso podría generar una nueva combinación por lo que se eliminan otra vez ciertas fichas y así sucesivamente.

Finalmente lo ultimo a considerar es el caso de eliminación de filas y columnas, dado que deberemos modificar el tamaño del “rectángulo” cuando sea necesario, siguiendo la regla del 65%.

DISEÑO:

Antes que nada, debemos tener en mente lo que se nos pidió en clase de presentar una solución modulada separando las diferentes responsabilidades del programa, “Divide y vencerás”.

El tablero será almacenado mediante un puntero a memoria dinámica compatible con bytes, por ejemplo: `unsigned char *tablero`

Este puntero representará el bloque físico de memoria en el que se almacenará la secuencia continua de bits. Los elementos individuales del arreglo no representarán fichas completas, sino fragmentos de la información empaquetada.

Ya a partir de esta información, lo que debemos hacer es diseñar funciones que vamos a tener que implementar en el main de nuestro juego, encargadas de permitir obtener el valor de una ficha, modificar una nueva ficha, generar una nueva ficha, etc.

Para terminar este informe me gustaría adjuntar 2 imágenes de análisis que hice en mi cuaderno, una seria de la lógica de pasar de la posición lógica a los bits de la ficha y la otra seria del flujo general del programa:

Tamaño del tablero en memoria para un tablero $F \times C$
 existen $F \times C$ - posiciones, la cantidad mínima de
 bits sera $F \times C \times 3 \rightarrow$ bytes: $3 \times F \times C / 8$ redondeando hacia arriba

La memoria reservada debiera corresponder a las
 dimensiones **ACTUALES** del tablero

tablero conceptual

	0	1	2	3	Columns
0					
1					
2					
3					
Filas					

$(Fila, columna) = (2, 3)$ Posición lógica

Indice: ¿Que ficha es? (0, 1, 2, 3, 4)

Fila $\times$ Columnas + Columna = 7

Bit inicial: Índice $\times 3 = 51$

Ahora queremos saber en que byte esta:

Sabemos que:

Byte 0: 0 - 7

Byte 1: 8 - 15

Byte 2: 16 - 23

Byte 3: 24 - 31

Byte 4: 32 - 39

Byte 5: 40 - 47

Byte 6: 48 - 55

→ aquí esta nuestra ficha

$$\text{byte} = \text{bit inicial} / 8 = 6$$

$$\text{posición en ese byte} = 51 \% 8 = 3$$

esta en el byte 6 en el bit 3

Posición lógica → Índice → Bit inicial → Byte correspondiente

↓
 Bits de la ficha

Flujo: el usuario establece la dimension

↓
Reserva de memoria

↓
Menu: eliminar ficha, fila o columna ←

↓
generar nuevas ficha

↓
deteccion nuevas combinaciones

↓
eliminar combinaciones

↓
Reorganiza. tablero

↓
Si — ¿Existen mas combinaciones? — No

